

Accounting Reports — Implementation Proposal

Generated: 2026-03-08 Sources: Implementation vault (domain-model.md, aggregates.md, read-models.md, isolation-acl.md, god-aggregate.md, message-map.md, bounded-contexts.md, project-architecture.md), planning vault (shoebox-offering.md, demo-requirements.md, demo-gap-analysis.md)

The Problem

The initial demo needs a better trial balance view in Excel. The required format is a simple, flat accounting table — but it doesn't match the model server's layout approach and can't be easily generated from it.

The trial balance is the first of a natural collection of **accounting reports** that share the same characteristic: they are **read-only projections** from existing events, rendered as flat Excel tables, with no dependency on the model server's table/row/formula infrastructure.

The Accounting Reports Collection

The trial balance is the first of a family of standard accounting reports. All share the same pattern:

- **Read-only projections** from existing event streams
- **Flat table format** — standard layouts every accountant recognizes
- **No model server involvement** — bypasses ServerFinancialModel entirely
- **ExcelDNA rendering** — written directly to dedicated Excel worksheets

Report Catalog

Group	Report	Format	Temporal Type	Source Streams	Priority
Financial Statements	Trial Balance	Account × Debit/Credit	As-of	\$ce-ChartOfAccounts, \$ce-AccountBalance, \$ce-EntrySet	P0 — initial demo
	Income Statement (P&L)	Revenue – Expenses = Net Income	Period	Same as Trial Balance	P1
	Balance Sheet	Assets = Liabilities + Equity	As-of	Same as Trial Balance	P1
Ledger Reports	General Ledger	All transactions by account, running balance	Period	\$ce-DataSource, \$ce-DataElement, \$ce-EntrySet, \$ce-ChartOfAccounts	P1
	Transaction Detail	All classified transactions with categories	Period	\$ce-DataSource, \$ce-DataElement, \$ce-CustomStatementTxType	P2

Receivables & Payables	AR Aging	Outstanding receivables by age bucket	As-of	\$ce-Counterparty, \$ce-DataElement (customer-linked)	P2
	AP Aging	Outstanding payables by age bucket	As-of	\$ce-Counterparty, \$ce-DataElement (vendor-linked)	P2
Tax	Income/Expense Summary	Categorized totals for tax preparation	Period	\$ce-ChartOfAccounts, \$ce-AccountBalance	P1

All reports derive from the same event streams already in the system. No new aggregates, no new commands, no expansion of `ServerFinancialModel`.

Why the Model Server Can't Do This

The `ServerFinancialModel` (3,557 lines, 87 Apply methods, 166 commands) is designed for **editable financial models** — tables with rows, formulas, data mappings, composite rows, and worksheets.

```
DataSource → DataTableDefinition → DataTableMap → ServerFinancialModel
  → rows with types (data/formula/composite/reference)
  → mapped to worksheets
    → rendered via ModelTemplateRm (82 events) + FinancialModelTablesRm (54 events)
```

Accounting reports need **none of this**. They are read-only, flat, standard-format, and point-in-time. Forcing them through the model server couples them to the god aggregate and its 166-command infrastructure for no benefit.

Architecture: Reports as Read Models

```
Existing event streams (no changes)
├─ $ce-ChartOfAccounts (account structure, categories)
├─ $ce-AccountBalance (balance snapshots)
├─ $ce-EntrySet (manual journal entries)
├─ $ce-DataSource / $ce-DataElement (imported data)
├─ $ce-Counterparty (contacts, roles)
└─ $ce-CustomStatementTxType (classifications)
↓
AccountingReportRm (per report – ReadModelBase in SpreadsheetAdapter)
  → subscribes to relevant category streams
  → builds typed projection (flat list of report lines)
  → implements IAccountingReport interface
  ↓
AccountingReportWriter (shared – Excel.Core)
  → reads report projection via IAccountingReport
  → creates/updates a dedicated Excel worksheet per report
  → standard formatting, headers, subtotals, number formats
  → managed by AccountingReportsContext (lifecycle, refresh)
```

Report Layouts

Standard accounting reports use two distinct layout patterns:

Single panel — sections stacked sequentially (Trial Balance, GL, Transaction Detail, Income/Expense Summary):

Trial Balance			
Assets			
1000	Cash	12,450	-
1100	AR	3,200	-
	Subtotal	15,650	-
Liabilities			
2000	AP	-	2,100
	Subtotal	-	2,100
...			
Totals		20,600	20,600

Two panels — placed side by side (Balance Sheet, possibly Income Statement):

ASSETS		LIABILITIES	
Current Assets		Current Liabilities	
Cash	12,450	AP	2,100
AR	3,200	Accrued	900
Subtotal	15,650	Subtotal	3,000
Fixed Assets		EQUITY	
Equipment	5,000	Owner's Equity	10,000
Subtotal	5,000	Retained Earn	7,650
		Subtotal	17,650
TOTAL ASSETS		TOTAL L+E	20,650

Common Interface

```
public interface IAccountingReport
{
    string ReportName { get; } // "Trial Balance", "Income Statement", etc.
    string SheetName { get; } // "Trial Balance", "P&L", "Balance Sheet"
    ReportGroup Group { get; } // FinancialStatements, Ledger, Receivables, Tax
    ReportTemporalType TemporalType { get; } // determines which ReportPeriod subtype
    applies
    IReadOnlyList<IReportPanel> Panels { get; } // 1 panel = stacked, 2 panels = side-by-
    side
    ReportPeriod Period { get; } // the time context for this report
}
```

```

    bool HasData { get; }
}

// — Time period model —————
public enum ReportTemporalType
{
    AsOf,        // "As of Dec 31, 2025" – balance snapshot at a point in time
    Period       // "For the month ended Dec 31, 2025" – activity over a date range
}

public abstract record ReportPeriod(string DisplayLabel)
{
    // Point-in-time: balances as of this date
    public sealed record AsOf(DateTime Date)
        : ReportPeriod($"As of {Date:MMMM d, yyyy}");

    // Date range: activity between From and Through
    public sealed record DateRange(DateTime From, DateTime Through)
        : ReportPeriod($"For the period {From:MMM d} \u2013 {Through:MMM d, yyyy}");
}

public enum ReportGroup
{
    FinancialStatements,
    Ledger,
    ReceivablesPayables,
    Tax
}

// A panel is one vertical column of sections.
// The panel count determines the layout:
//   1 panel → rendered top-to-bottom (Trial Balance, GL, etc.)
//   2 panels → rendered side-by-side with a gap (Balance Sheet, etc.)
public interface IReportPanel
{
    string PanelTitle { get; }           // "ASSETS", "LIABILITIES & EQUITY", or null
    IReadOnlyList<ReportColumn> Columns { get; }
    IReadOnlyList<IReportSection> Sections { get; }
    object[] TotalRow { get; }          // Panel-level total (e.g., "TOTAL ASSETS")
}

public record ReportColumn(string Header, Type DataType, string NumberFormat);

public interface IReportSection
{
    string SectionName { get; }          // "Current Assets", "Revenue", etc.
    IReadOnlyList<object[]> Rows { get; }
    object[] SubtotalRow { get; }        // Section subtotal
}

```

The layout is determined entirely by the **panel count** — no separate layout enum:

- **1 panel** → writer renders sections top-to-bottom
- **2 panels** → writer places them side by side with a gap between column ranges

The `AccountingReportWriter` checks `report.Panels.Count` and branches accordingly. Every report read model implements `IAccountingReport` — the writer doesn't need to know which report it's rendering.

Read Model Internals: State → Projection

Every accounting report RM has two distinct concerns:

1. **State accumulation** — event handlers build raw dictionaries from event streams
2. **Report projection** — a declarative mapping transforms raw state into the `IAccountingReport` structure on demand

The mapping is the part a developer writes when translating a report layout image or Excel example into code. It should read like the visual layout itself.

State Accumulation (Shared Base)

Most accounting reports draw from the same underlying data. A shared base class accumulates this state so individual report RMs don't repeat it:

```
public abstract class AccountingReportBase : ReadModelBase, IAccountingReport
{
    // — Raw state populated by event handlers —————
    protected readonly Dictionary<Guid, AccountRecord> Accounts = new();
    protected readonly List<TransactionRecord> Transactions = new();

    // — Common event handlers —————
    // ChartOfAccounts events → Accounts dictionary
    void Handle(RootAccountAdded e)
        => Accounts[e.AccountId] = new(e.AccountId, e.Number, e.Name, e.Category, ...);
    void Handle(ChildAccountAdded e)
        => Accounts[e.AccountId] = new(..., ParentId: e.ParentAccountId);
    void Handle(AccountUpdated e)
        => Accounts[e.AccountId] = Accounts[e.AccountId] with { Name = e.Name, ... };

    // EntrySet / DataElement events → Transactions list (all with dates)
    void Handle(ManualEntryAdded e)
        => Transactions.Add(new(e.AccountId, e.Amount, e.Date, e.Description, ...));
    void Handle(DataElementClassified e)
        => Transactions.Add(new(e.AccountId, e.Amount, e.TransactionDate, ...));

    // — Temporal query helpers —————

    // Balance as of a date = sum of all transactions through that date
    protected decimal BalanceAsOf(Guid accountId, DateTime asOf)
        => Transactions
            .Where(t => t.AccountId == accountId && t.Date <= asOf)
            .Sum(t => t.Amount);

    // Activity for a period = sum of transactions in date range
```

```

protected decimal ActivityForPeriod(Guid accountId, DateTime from, DateTime through)
=> Transactions
    .Where(t => t.AccountId == accountId && t.Date >= from && t.Date <= through)
    .Sum(t => t.Amount);

// All transactions in a date range (for GL/detail reports)
protected IEnumerable<TransactionRecord> TransactionsInPeriod(DateTime from, DateTime
through)
=> Transactions.Where(t => t.Date >= from && t.Date <= through);

// — Abstract: each report defines its own projection —
public abstract IAccountingReport BuildReport(ReportPeriod period);

// — Shared records —
protected record AccountRecord(
    Guid AccountId, string Number, string Name,
    AccountingCategory Category, AccountSubtype Subtype,
    Guid? ParentId);

protected record TransactionRecord(
    Guid AccountId, decimal Amount, DateTime Date,
    string Description, string Source);
}

```

A report RM inherits `AccountingReportBase` and only needs to:

1. Declare which streams to subscribe to (override `Start()`)
2. Define the mapping from accumulated state → report layout

Declarative Report Mapping

The mapping is a fluent builder that a developer writes by looking at the target layout. The builder produces the `IAccountingReport` projection on demand.

```

public class ReportBuilder
{
    // — Entry points (match the layout type) —
    public static VerticalReportBuilder Vertical(string name, string sheet);
    public static SideBySideReportBuilder SideBySide(string name, string sheet);
}

```

Vertical Builder

```

public class VerticalReportBuilder
{
    // Define columns — each lambda extracts one cell from a row source object
    VerticalReportBuilder Column<T>(string header, Func<T, object> accessor, string format =
null);

    // Define a section — filter + optional sort determine which rows appear
    VerticalReportBuilder Section<T>(

```

```

        string name,
        IEnumerable<T> rows,
        Func<T, object>[] subtotalAggregates = null); // per-column aggregation

// Grand total across all sections
VerticalReportBuilder GrandTotal(string label, params Func<IEnumerable<object>[],
object>[] aggregates);

// Produce the final projection
IAccountingReport Build(ReportPeriod period);
}

```

Side-by-Side Builder

```

public class SideBySideReportBuilder
{
    // Each panel is its own vertical layout
    SideBySideReportBuilder LeftPanel(string title, Action<PanelBuilder> configure);
    SideBySideReportBuilder RightPanel(string title, Action<PanelBuilder> configure);
    IAccountingReport Build(ReportPeriod period);
}

public class PanelBuilder
{
    PanelBuilder Column<T>(string header, Func<T, object> accessor, string format = null);
    PanelBuilder Section<T>(string name, IEnumerable<T> rows, ...);
    PanelBuilder PanelTotal(string label, ...);
}

```

Worked Example: Trial Balance

A developer sees this target layout (from an image or Excel file):

TRIAL BALANCE – Acme Corp – As of Dec 31, 2025			
Acct #	Account Name	Debit	Credit
ASSETS			
1000	Cash - Checking	12,450.00	
1100	Accounts Receivable	3,200.00	
	Assets Subtotal	15,650.00	
LIABILITIES			
2000	Accounts Payable		2,100.00
	Liabilities Subtotal		2,100.00
EQUITY			
3000	Owner's Equity		10,000.00

...		
TOTALS	20,600.00	20,600.00

And writes this mapping in the report RM:

```
public class TrialBalanceRm : AccountingReportBase
{
    public override ReportTemporalType TemporalType => ReportTemporalType.AsOf;

    public TrialBalanceRm(/* bus, repo */) : base(/* ... */)
    {
        Start<ChartOfAccounts>();
        Start<EntrySet>();
    }

    // — The mapping: visual layout → code —————
    public override IAccountingReport BuildReport(ReportPeriod period)
    {
        var asOf = ((ReportPeriod.AsOf)period).Date;
        var lines = BuildTrialBalanceLines(asOf); // state → flat lines

        return ReportBuilder
            .Vertical("Trial Balance", "Trial Balance")
            .InGroup(ReportGroup.FinancialStatements)

            // Columns – left to right, exactly as the layout shows
            .Column("Acct #", (TBLLine l) => l.AccountNumber)
            .Column("Account Name", (TBLLine l) => l.AccountName)
            .Column("Debit", (TBLLine l) => l.Debit, "$#,##0.00")
            .Column("Credit", (TBLLine l) => l.Credit, "$#,##0.00")

            // Sections – top to bottom, exactly as the layout shows
            .Section("Assets",
                lines.Where(l => l.Category == AccountingCategory.Asset),
                subtotal: l => (null, null, l.Sum(x => x.Debit), l.Sum(x => x.Credit)))

            .Section("Liabilities",
                lines.Where(l => l.Category == AccountingCategory.Liability),
                subtotal: l => (null, null, l.Sum(x => x.Debit), l.Sum(x => x.Credit)))

            .Section("Equity",
                lines.Where(l => l.Category == AccountingCategory.Equity),
                subtotal: l => (null, null, l.Sum(x => x.Debit), l.Sum(x => x.Credit)))

            .Section("Income",
                lines.Where(l => l.Category == AccountingCategory.Income),
                subtotal: l => (null, null, l.Sum(x => x.Debit), l.Sum(x => x.Credit)))

            .Section("Expenses",
                lines.Where(l => l.Category == AccountingCategory.Expense),
```



```

        subtotal: l => (null, null, l.Sum(x => x.Debit), l.Sum(x => x.Credit)))

// Grand total – the bottom line
.GrandTotal("TOTALS",
    all => (null, null, all.Sum(x => x.Debit), all.Sum(x => x.Credit)))

.Build(period);
}

// — State → flat lines (debit/credit assignment) —————
// Uses BalanceAsOf() from base class – pass a different date, get a different TB
private IReadOnlyList<TBLine> BuildTrialBalanceLines(DateTime asOf)
{
    return Accounts.Values
        .Select(a =>
        {
            var balance = BalanceAsOf(a.AccountId, asOf);
            var isDebitNormal = a.Category is
                AccountingCategory.Asset or
                AccountingCategory.Expense or
                AccountingCategory.Dividend;

            return new TBLine(
                a.Number, a.Name, a.Category,
                Debit: isDebitNormal ? Math.Max(balance, 0) : Math.Max(-balance, 0),
                Credit: isDebitNormal ? Math.Max(-balance, 0) : Math.Max(balance, 0));
        })
        .Where(l => l.Debit != 0 || l.Credit != 0) // skip zero-balance accounts
        .OrderBy(l => l.Category)
        .ThenBy(l => l.AccountNumber)
        .ToList();
}

private record TBLine(
    string AccountNumber, string AccountName,
    AccountingCategory Category,
    decimal Debit, decimal Credit);
}

```

The mapping reads like the layout:

- Columns listed left to right match the visual columns
- Sections listed top to bottom match the visual sections
- Subtotal lambdas define what goes in the subtotal row
- Grand total at the end matches the bottom line

A developer holds the layout image on one screen and the mapping code on the other — the correspondence is 1:1.

Worked Example: Balance Sheet (Side-by-Side)

Target layout:

ASSETS		LIABILITIES & EQUITY	
Current Assets		Current Liabilities	
Cash	12,450	AP	2,100
AR	3,200	Accrued	900
Subtotal	15,650	Subtotal	3,000
Fixed Assets		Equity	
Equipment	5,000	Owner's Eq	10,000
Subtotal	5,000	Retained	7,650
		Subtotal	17,650
TOTAL ASSETS	20,650	TOTAL L+E	20,650

Mapping:

```
public class BalanceSheetRm : AccountingReportBase
{
    public override ReportTemporalType TemporalType => ReportTemporalType.AsOf;

    public override IAccountingReport BuildReport(ReportPeriod period)
    {
        var asOf = ((ReportPeriod.AsOf)period).Date;
        var accounts = BuildBalanceLines(asOf);

        return ReportBuilder
            .SideBySide("Balance Sheet", "Balance Sheet")
            .InGroup(ReportGroup.FinancialStatements)

            .LeftPanel("ASSETS", panel => panel
                .Column("Account", (BSLine l) => l.AccountName)
                .Column("Amount", (BSLine l) => l.Balance, "$#,##0.00")

                .Section("Current Assets",
                    accounts.Where(l => l.Subtype == AccountSubtype.CurrentAsset))
                .Section("Fixed Assets",
                    accounts.Where(l => l.Subtype == AccountSubtype.FixedAsset))
                .Section("Other Assets",
                    accounts.Where(l => l.Subtype == AccountSubtype.OtherAsset))

                .PanelTotal("TOTAL ASSETS"))

            .RightPanel("LIABILITIES & EQUITY", panel => panel
                .Column("Account", (BSLine l) => l.AccountName)
                .Column("Amount", (BSLine l) => l.Balance, "$#,##0.00")

                .Section("Current Liabilities",
                    accounts.Where(l => l.Subtype == AccountSubtype.CurrentLiability))
                .Section("Long-Term Liabilities",
```

```

        accounts.Where(1 => 1.Subtype == AccountSubtype.LongTermLiability))
        .Section("Equity",
            accounts.Where(1 => 1.Category == AccountingCategory.Equity))

        .PanelTotal("TOTAL LIABILITIES & EQUITY"))

    .Build(period);
}
}

```

Again — left panel in code = left side of layout, right panel = right side. Sections in each panel match top-to-bottom.

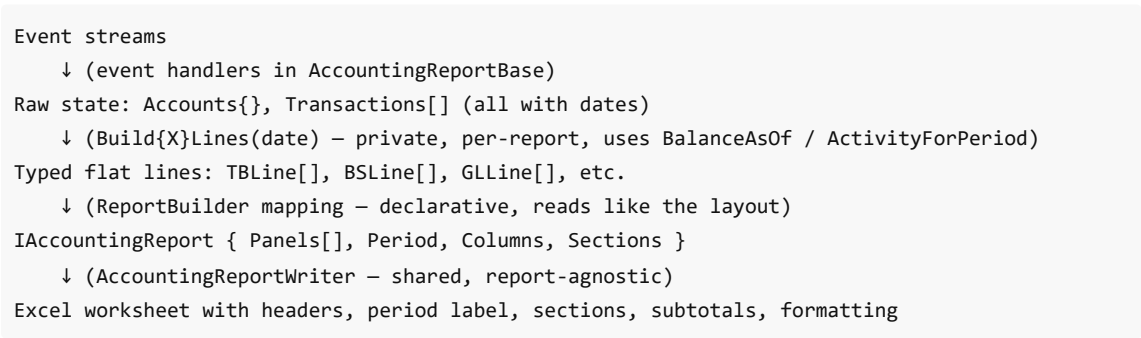
The Developer Workflow

When adding a new report, the developer:

1. **Gets the layout** — an image, a screenshot, an Excel file, or a hand-drawn sketch
2. **Creates the RM class** inheriting `AccountingReportBase`
3. **Declares stream subscriptions** — which `Start<TAgg>()` calls (most reports reuse the same 2–3 streams)
4. **Writes the mapping** by reading the layout:
 - Is it vertical or side-by-side? → `ReportBuilder.Vertical(...)` or `.SideBySide(...)`
 - What columns? → `.Column(...)` calls, left to right
 - What sections? → `.Section(...)` calls, top to bottom
 - What subtotals/totals? → aggregation lambdas
5. **Writes `Build{X}Lines(date)`** — the private method that uses temporal helpers (`BalanceAsOf` , `ActivityForPeriod`) to transform raw state into the report's row type
6. **Registers in `AccountingReportsContext`** — one line

Steps 3–5 are typically **under 80 lines of code**. The builder handles all structural concerns (producing `IReportPanel` , `IReportSection` , etc.). The writer handles all Excel concerns. The developer only thinks about: "what data goes where on the layout."

How the Pieces Connect



Layer	Who writes it	How often it changes
Event handlers	Framework (AccountingReportBase)	Once — when new event types are added
Build{X}Lines	Report developer	Once per report — simple LINQ over state

ReportBuilder mapping	Report developer	Once per report — matches the visual layout
AccountingReportWriter	Framework	Once — handles all reports generically

Adding a New Report (Pattern)

Each new report is three steps:

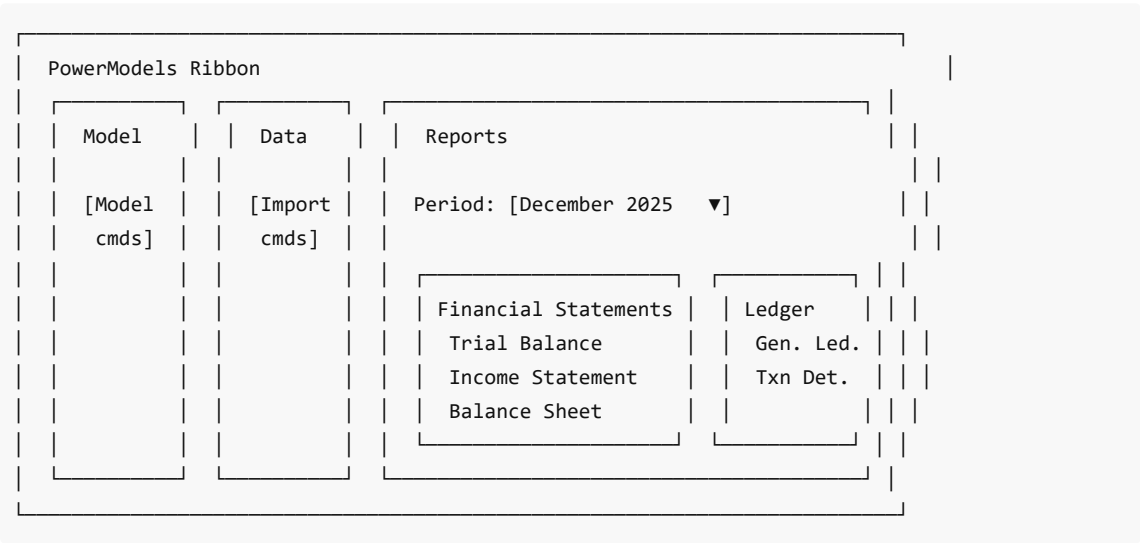
- 1. Create {Report}Rm in SpreadsheetAdapter/ReadModels/AccountingSystem/ — inherit AccountingReportBase , declare streams, write mapping + lines method (~80 lines)
- 2. Register in AccountingReportsContext — one line
- 3. Ribbon auto-discovers — the report appears in the Reports ribbon group

No changes to the model server, no new commands, no aggregate modifications, no changes to the writer or framework.

UI: How Reports Are Grouped and Enabled

Excel Ribbon — "Reports" Group

The ExcelDNA add-in already has a ribbon with a RibbonContextUpdated event for managing state. Reports add a new Reports group (or tab) to the existing ribbon, including a period selector:



Period Selector

The period dropdown is shared across all reports. When the user selects a period, the AccountingReportsContext maps it to the correct ReportPeriod subtype per report:

Period Selection	As-of Reports (TB, BS, Aging)	Period Reports (P&L, GL, Tax)
December 2025	AsOf(Dec 31, 2025)	DateRange(Dec 1 - Dec 31, 2025)
Q4 2025	AsOf(Dec 31, 2025)	DateRange(Oct 1 - Dec 31, 2025)
FY 2025	AsOf(Dec 31, 2025)	DateRange(Jan 1 - Dec 31, 2025)

YTD	AsOf(today)	DateRange(fiscal year start - today)
Custom...	AsOf(end date)	DateRange(start - end)

The context uses `report.TemporalType` to decide which subtype to construct:

```
public ReportPeriod ResolvePeriod(IAccountingReport report, SelectedPeriod selection)
{
    return report.TemporalType switch
    {
        ReportTemporalType.AsOf => new ReportPeriod.AsOf(selection.EndDate),
        ReportTemporalType.Period => new ReportPeriod.DateRange(selection.StartDate,
selection.EndDate),
    };
}
```

For P0 (initial demo): Default to current month. The dropdown can be added later — the interface supports it from day one, but the UI just passes `AsOf(DateTime.Today)` .

Enablement Rules

Reports are **contextual** — they enable/disable based on business state:

Condition	Reports Group State
No workspace open	Disabled (grayed out)
Workspace open, no business selected	Disabled
Business selected, no ChartOfAccounts	Disabled with tooltip: "Set up Chart of Accounts first"
Business selected, ChartOfAccounts exists	Enabled — all applicable reports available
Report has no data (e.g., no AR transactions)	Individual button disabled with tooltip

This follows the existing pattern — `RibbonContextUpdated` fires when the business context changes, and the ribbon callback checks `IAccountingReport.HasData` for each report.

Worksheet Management

Each report gets a **dedicated worksheet** in the workbook:

[Model Sheet 1] [Model Sheet 2] [Trial Balance] [P&L] [GL]

↑ report sheets

Behavior:

- **First click:** Creates a new worksheet named after the report, writes the report data
- **Subsequent clicks:** Navigates to the existing report worksheet and refreshes data
- **Report worksheets are protected** — read-only (no accidental edits to report output)

- **Tab color convention:** Report worksheets get a distinct tab color (e.g., blue) to visually separate from model worksheets
- **Auto-refresh:** Report worksheets subscribe to live events via a `TransientSubscriber` wrapper — when underlying data changes, the report updates automatically

AccountingReportsContext

Location: `src/Excel/PowerModels.Excel.Core/Reports/AccountingReportsContext.cs`

This is the lifecycle manager for all accounting reports in a business context:

```
public class AccountingReportsContext
{
    private readonly Dictionary<string, IAccountingReport> _reports;
    private readonly AccountingReportWriter _writer;

    // Called when a business context is opened
    public void Initialize(ModelServerReadContext readContext)
    {
        // Wire up all registered report RMs to the business event streams
        // Each RM starts receiving events and building its projection
    }

    // Called from ribbon button click
    public void ShowReport(string reportName)
    {
        var report = _reports[reportName];
        _writer.WriteToWorksheet(report); // creates or refreshes the worksheet
    }

    // Called on RibbonContextUpdated
    public IEnumerable<(string Name, ReportGroup Group, bool Enabled)> GetAvailableReports()
    {
        return _reports.Values.Select(r => (r.ReportName, r.Group, r.HasData));
    }

    // Called when business context closes
    public void Dispose()
    {
        // Unsubscribe all report RMs, remove report worksheets if desired
    }
}
```

Registration pattern:

```
// In AccountingReportsContext constructor or factory
_reports.Add("Trial Balance", new TrialBalanceRm(bus, repository));
_reports.Add("Income Statement", new IncomeStatementRm(bus, repository));
_reports.Add("Balance Sheet", new BalanceSheetRm(bus, repository));
// ... each new report is one line
```

AccountingReportWriter

Location: src/Excel/PowerModels.Excel.Core/Reports/AccountingReportWriter.cs

Shared writer that renders any `IAccountingReport` to an Excel worksheet:

```
public class AccountingReportWriter
{
    public void WriteToWorksheet(IAccountingReport report)
    {
        // 1. Find or create worksheet named report.SheetName
        // 2. Clear existing content
        // 3. Write report header (name, as-of date, business name)
        // 4. Write column headers from report.Columns
        // 5. For each section in report.Sections:
        //     a. Write section header (bold, e.g., "Assets")
        //     b. Write section rows
        //     c. Write subtotal row (bold, bordered)
        // 6. Write grand total row
        // 7. Apply number formats from report.Columns
        // 8. Auto-fit columns
        // 9. Set worksheet protection (read-only)
        // 10. Set tab color (report color)
    }
}
```

This writer is **report-agnostic**. Adding a new report type requires zero changes to the writer.

What This Doesn't Touch

- ServerFinancialModel (0 changes)
- FinancialModelService (0 changes)
- ModelTemplateRm, FinancialModelTablesRm, or any existing RM (0 changes)
- Any existing table/row/formula/mapping infrastructure (0 changes)
- SpreadsheetContextService routing (reports don't use command flow)

What This Adds

- `IAccountingReport` interface + `ReportGroup` enum (shared contract)
- `AccountingReportsContext` (lifecycle manager, one per business)
- `AccountingReportWriter` (shared Excel writer, report-agnostic)
- Per-report read models (one `ReadModelBase` each, in `SpreadsheetAdapter`)
- Ribbon group with contextual enablement
- Dedicated report worksheets with auto-refresh

Estimated Effort

Component	Complexity	Estimate
IAccountingReport + shared types	Low — interface + records	0.5 day

AccountingReportWriter	Medium — generic Excel writer with formatting	1–2 days
AccountingReportsContext	Low — lifecycle + registration	0.5–1 day
Ribbon "Reports" group + enablement	Low — follows existing ribbon pattern	0.5–1 day
TrialBalanceRm (P0)	Low — 3 streams, flat projection	1–2 days
Total for infrastructure + Trial Balance		4–6 days
Each subsequent report	Low — new RM implementing IAccountingReport	1–2 days each

Once the infrastructure is in place (writer, context, ribbon group), each new report is just a new read model + one registration line. The pattern scales to the full catalog without additional framework work.

Checkpoint/Persistence

Report RMs are `ReadModelBase` — they replay from position 0 on startup. The streams they subscribe to (`$ce-ChartOfAccounts`, `$ce-AccountBalance`, `$ce-EntrySet`) are low-volume (1–2 existing subscribers each), so marginal cost is small.

Recommendation: Wire in `ICheckpointStore` from day one if building during or after the mitigation plan. This is the pattern being established for all RMs anyway. Report RMs checkpoint their position, resume from last checkpoint on restart.

Summary

Accounting reports are **projections, not models**. They belong in the read model layer, rendered directly to Excel via ExcelDNA, with no model server involvement.

The implementation creates a **reusable framework** — shared interface, shared writer, shared ribbon group — so the trial balance (P0) establishes the pattern and every subsequent report is incremental (1–2 days each).

	Model Server Tables	Accounting Reports
Purpose	Editable financial models	Read-only standard reports
Data flow	DataSource → Map → SFM → RM → Excel	Event streams → Report RM → Excel
Aggregate	ServerFinancialModel (god aggregate)	None — pure read models
Commands	166 via FinancialModelService	0 — no write path
Excel output	Model worksheet with formulas	Dedicated report worksheet (protected)
Adding a new one	Weeks of work touching SFM	1–2 days: new RM + register

Appendix A: User-Defined Report Templates (Future Expansion)

This is a separate concern from the P0 implementation. The core framework uses C# report RMs written by developers. This appendix explores how the same framework could later be driven by user-defined templates —

enabling accountants and firm administrators to create and customize reports without developer involvement.

The Opportunity

The `ReportBuilder` fluent DSL already reads like a layout description. The step from code to data is short:

```
Code:      .Column("Acct #", (TBLine l) => l.AccountNumber)
Data:      { header: "Acct #", field: "AccountNumber" }
```

If the report definition becomes data instead of code, it can be authored by users through a UI or a structured file — and interpreted at runtime by a single generic `TemplateReportRm`.

Why This Matters for Firms

PowerModels' firm-first GTM means firms are the primary customer. Firms have strong opinions about report layouts:

- "We always present the P&L with these category groupings"
- "Our Balance Sheet uses this specific section order"
- "We need a custom report that combines AR aging with income by client"

Today: every custom report requires a developer to write an RM. At scale (hundreds of firms, each with preferences), this doesn't work.

With templates: a firm admin designs their report layout once, and it applies to all their clients. This is a **product feature**, not developer tooling.

Template Definition Format

A report template is a YAML/JSON document that mirrors the `ReportBuilder` DSL:

```
# Trial Balance template – equivalent to the C# TrialBalanceRm
name: Trial Balance
sheet: Trial Balance
group: FinancialStatements
temporal: AsOf

dataSources:
  - ChartOfAccounts      # maps to Start<ChartOfAccounts>()
  - EntrySet              # maps to Start<EntrySet>()

lineType:
  fields:
    - name: AccountNumber
      source: account.Number
    - name: AccountName
      source: account.Name
    - name: Category
      source: account.Category
    - name: Balance
      source: balanceAsOf(account.Id, period.date)
  computed:
    - name: Debit
      expr: "isDebitNormal(Category) ? max(Balance, 0) : max(-Balance, 0)"
    - name: Credit
```

```

    expr: "isDebitNormal(Category) ? max(-Balance, 0) : max(Balance, 0)"

panels:
- columns:
  - header: "Acct #"
    field: AccountNumber
  - header: "Account Name"
    field: AccountName
  - header: "Debit"
    field: Debit
    format: "$#,##0.00"
  - header: "Credit"
    field: Credit
    format: "$#,##0.00"

sections:
- name: Assets
  filter: "Category == Asset"
  sort: AccountNumber
  subtotal: [null, null, "sum:Debit", "sum:Credit"]

- name: Liabilities
  filter: "Category == Liability"
  sort: AccountNumber
  subtotal: [null, null, "sum:Debit", "sum:Credit"]

- name: Equity
  filter: "Category == Equity"
  sort: AccountNumber
  subtotal: [null, null, "sum:Debit", "sum:Credit"]

- name: Income
  filter: "Category == Income"
  sort: AccountNumber
  subtotal: [null, null, "sum:Debit", "sum:Credit"]

- name: Expenses
  filter: "Category == Expense"
  sort: AccountNumber
  subtotal: [null, null, "sum:Debit", "sum:Credit"]

grandTotal:
  label: "TOTALS"
  values: [null, null, "sum:Debit", "sum:Credit"]

```

A side-by-side Balance Sheet template would have two entries under `panels:` instead of one — the panel count still determines the layout.

How It Works

```

Template (YAML/JSON)
  ↓ (parsed at runtime)
TemplateReportDefinition (in-memory representation)
  ↓ (fed into)
TemplateReportRm : AccountingReportBase
  → subscribes to dataSources streams
  → on BuildReport(period):
    1. evaluates lineType over Accounts + Transactions using temporal helpers
    2. applies filters/sorts per section
    3. computes subtotals via aggregation expressions
    4. produces IAccountingReport via the same ReportBuilder DSL internally
  ↓
AccountingReportWriter (unchanged – still report-agnostic)
  ↓
Excel worksheet

```

Key insight: `TemplateReportRm` is the only new RM. It's a generic report RM parameterized by a template definition. Every user-defined report is an instance of `TemplateReportRm` with a different template — no new C# classes.

Expression Language

Templates need a safe, limited expression language for filters and computed fields. This is NOT arbitrary code execution — it's a constrained DSL over known fields:

Expression	Meaning
<code>Category == Asset</code>	Enum equality filter
<code>Subtype == CurrentAsset</code>	Subtype filter
<code>isDebitNormal(Category)</code>	Built-in function: true for Asset, Expense, Dividend
<code>balanceAsOf(accountId, date)</code>	Built-in temporal query from base class
<code>activityForPeriod(accountId, from, through)</code>	Built-in temporal query
<code>max(value, 0)</code>	Standard math
<code>sum:FieldName</code>	Aggregation in subtotal context

This is a **whitelist** of known operations over known data, not a general-purpose scripting language. It can be evaluated safely without sandboxing concerns.

Template Authoring: Three Tiers

Tier 1: Built-in templates (shipped with product)

- The standard reports (Trial Balance, P&L, Balance Sheet, GL, etc.)
- Defined in YAML, bundled as embedded resources
- Replace the hand-coded C# RMs over time (or coexist)
- Maintained by PowerModels team

Tier 2: Firm templates (created by firm admins)

- Firm admin customizes a built-in template or creates from scratch
- Stored per-firm in the data store
- Applied to all clients under that firm
- Authoring UI: structured form or YAML editor (not Excel)

Tier 3: Visual template editor (long-term)

- Drag-and-drop report builder in the UI
- Select data sources, define columns, drag sections, set filters
- Generates YAML under the hood
- Preview renders the report with sample data
- This is the "report designer" product feature

Migration Path from Code to Templates

The transition is gradual and non-breaking:

Phase	Reports Defined By	When
P0 (now)	C# report RMs via ReportBuilder DSL	Initial demo + MVP
Post-MVP	Built-in YAML templates replace C# RMs one-by-one	After product launch
v2	Firm admin template customization	Firm onboarding feature
v3	Visual template editor	Product maturity

At every phase, the downstream is identical: `IAccountingReport` → `AccountingReportWriter` → Excel. The writer never changes. The ribbon never changes. Only the source of the report definition evolves.

Template Storage

Tier	Storage	Scope
Built-in	Embedded YAML resources in assembly	Global — all firms
Firm	Event-sourced: <code>ReportTemplate</code> aggregate per firm	Per-firm, versioned
Client override	Optional per-client customization	Per-client

Making firm templates event-sourced means:

- Template changes are auditable (who changed what, when)
- Templates can be versioned and rolled back
- Template events feed into the same infrastructure
- A `FirmReportTemplatesRm` read model exposes available templates per firm

What the Template Approach Enables

Capability	Code-Only	With Templates
New standard report	Developer writes RM (1–2 days)	Developer writes YAML (hours)
Firm-specific layout	Developer writes custom RM	Firm admin customizes template
Client-specific tweak	Not feasible at scale	Client override on firm template

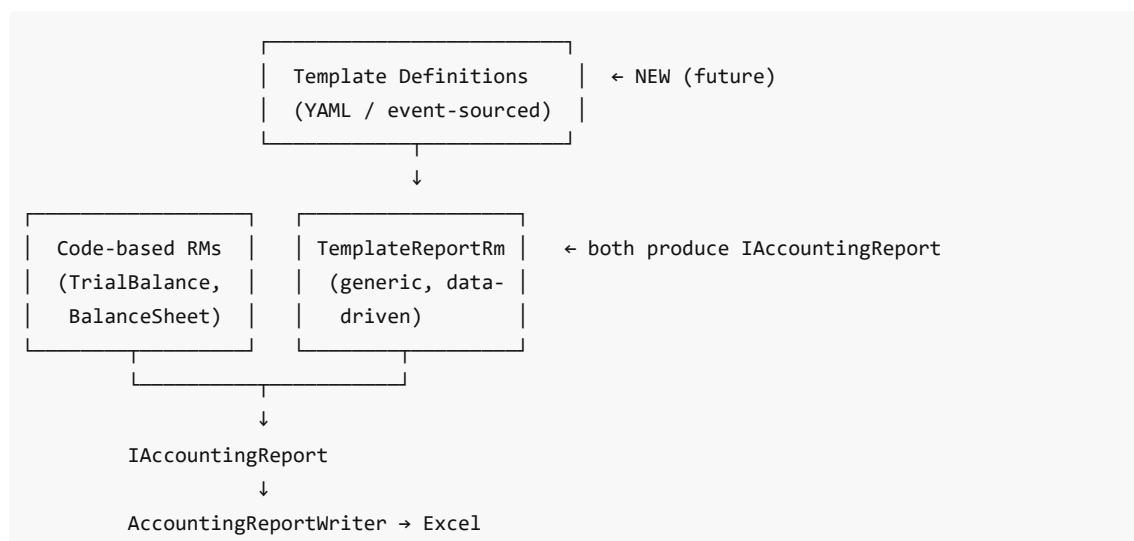
Non-dev contribution	Not possible	Accountants define reports
Report versioning	Git commits	Event-sourced template history
A/B testing layouts	Multiple RMs	Multiple template versions

Risks and Constraints

- **Expression language complexity** — Must stay constrained. If it creeps toward Turing-completeness, it becomes a maintenance and security liability. The whitelist approach prevents this.
- **Performance** — Template evaluation adds interpretation overhead vs compiled C#. For small reports (hundreds of rows) this is negligible. For large GL reports (thousands of transactions), may need optimization.
- **Error handling** — Bad templates need clear error messages ("Section filter references unknown field 'Categori'"). Validation at template save time, not render time.
- **Not needed for P0** — This is a post-MVP investment. The C# `ReportBuilder` DSL is the right approach for now. Templates are the scaling strategy.

Relationship to Existing Architecture

Templates don't change the architecture — they add a data-driven layer on top:



Everything below `IAccountingReport` is unchanged. The framework built for P0 is the same framework that templates plug into later.